

2024 DL Lab3 Machine Translation

Parameter size: **84695.364k**

The parameter size of transformer is **84695.364 k**

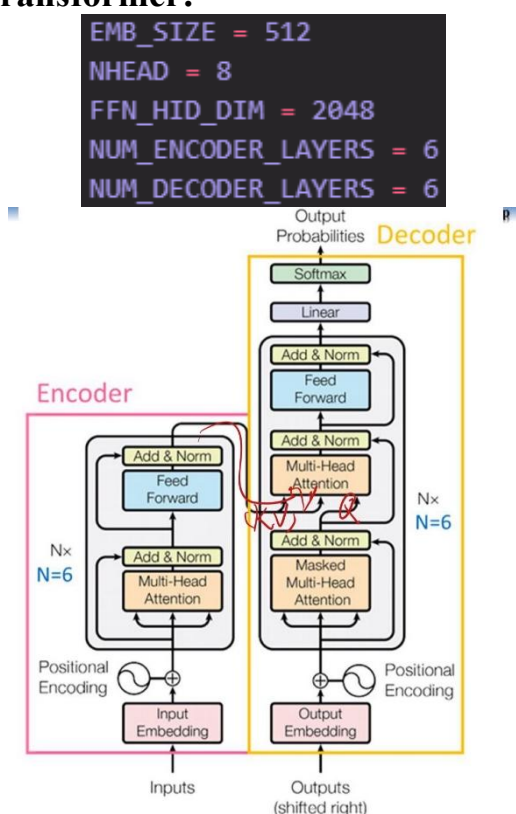
Accuracy(BLEU score): **0.290**

```
Epoch: 18, Train loss: 1.964, Val loss: 2.201, Val BLEU: 0.276, Epoch time = 348.189s
Model saved
Epoch: 19, Train loss: 1.943, Val loss: 2.205, Val BLEU: 0.278, Epoch time = 348.192s
Epoch: 20, Train loss: 1.924, Val loss: 2.210, Val BLEU: 0.280, Epoch time = 348.167s
Epoch: 21, Train loss: 1.905, Val loss: 2.209, Val BLEU: 0.282, Epoch time = 348.102s
Epoch: 22, Train loss: 1.886, Val loss: 2.212, Val BLEU: 0.286, Epoch time = 348.182s
Epoch: 23, Train loss: 1.868, Val loss: 2.217, Val BLEU: 0.288, Epoch time = 348.120s
Epoch: 24, Train loss: 1.850, Val loss: 2.224, Val BLEU: 0.289, Epoch time = 348.227s
Epoch: 25, Train loss: 1.833, Val loss: 2.233, Val BLEU: 0.290, Epoch time = 348.157s
```

Translation testing:

```
Input:           : 你好，欢迎来到中国
Prediction        : Hello, welcome China to go to China.
Ground truth      : Hello, Welcome to China
Bleu Score (1gram): 0.5714285969734192
Bleu Score (2gram): 0.30860671401023865
Bleu Score (3gram): 0.0
Bleu Score (4gram): 0.0
```

The structure of Transformer:



參考教授上課講義的，因此我 encoder 與 decoder 都各使用 6 層。

The problem I encountered:

在 training 時，我的 Train loss, Val loss 和 Val BLEU 都有不錯的結果，但是在 translation testing 始終沒有結果。我認為我的 Source Mask and Padding 可能與 Inference 沒有匹配到，因此我將 Inference 的部分如 src,src_mask,memory,out 等等 print 出來看看哪邊出問題，最終發現事實也使如此。最後我將 ys, tgt_padding_mask 和 src_mask，torch.ones()改成 torch.zeros()即可運作。

Inference:

```
def generate_square_subsequent_mask(sz):
    mask = (torch.triu(torch.ones(sz, sz)) == 1).transpose(0, 1)
    mask = mask.float().masked_fill(mask == 0, float('-1e20')).masked_fill(mask == 1, float(0.0))
    return mask

# function to generate output sequence using greedy algorithm
def greedy_decode(model, src, src_mask, max_len, start_symbol):
    src = src.to(DEVICE)
    src_mask = src_mask.to(DEVICE)
    memory = model.encode(src, src_padding_mask=src_mask)
    ys = torch.zeros(1, 1).fill_(start_symbol).type(torch.long).to(DEVICE)
    for i in range(max_len-1):
        memory = memory.to(DEVICE)
        tgt_padding_mask = torch.zeros(1, ys.size(0)).type(torch.bool).to(DEVICE) # target padding mask
        tgt_mask = (generate_square_subsequent_mask(ys.size(0))).type(torch.bool).to(DEVICE) # target causal mask
        out = model.decode(ys, memory, src_padding_mask=src_mask, tgt_padding_mask=tgt_padding_mask, tgt_future_mask=tgt_mask)
        out = out.transpose(0, 1)
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim=1)
        next_word = next_word.item()

        ys = torch.cat([ys, torch.ones(1, 1).type_as(src.data).fill_(next_word)], dim=0)
        if next_word == EOS_IDX:
            break
    return ys

# actual function to translate input sentence into target language
def translate(model: torch.nn.Module, src_sentence: str, input_tokenizer, output_tokenizer):
    model.eval()
    sentence = input_tokenizer.encode(src_sentence)
    sentence = torch.tensor(sentence).view(-1, 1)
    num_tokens = sentence.shape[0]

    src_mask = torch.zeros(1, num_tokens).type(torch.bool) # source padding mask
    tgt_tokens = greedy_decode(model, sentence, src_mask, max_len=num_tokens + 5, start_symbol=BOS_IDX).flatten()
    output_sentence = output_tokenizer.decode(tgt_tokens, skip_special_tokens=True)
    return output_sentence
```

Training strategy:

- Loss function 上面我選擇使用了 Label Smoothing Loss。

```
class LabelSmoothingLoss(nn.Module):
    def __init__(self, classes, smoothing=0.1, dim=-1, ignore_index=None):
        super(LabelSmoothingLoss, self).__init__()
        self.confidence = 1.0 - smoothing
        self.smoothing = smoothing
        self.cls = classes
        self.dim = dim
        self.ignore_index = ignore_index

    def forward(self, pred, target):
        pred = pred.log_softmax(dim=self.dim)
        with torch.no_grad():
            # 確保 target 是正確的數據類型和形狀
            target = target.long().view(-1, 1)

            true_dist = torch.zeros_like(pred)
            true_dist.fill_(self.smoothing / (self.cls - 1))
            if self.ignore_index is not None:
                true_dist[:, self.ignore_index] = 0
            true_dist.scatter_(1, target, self.confidence)
        return torch.mean(torch.sum(-true_dist * pred, dim=self.dim))

criterion = LabelSmoothingLoss(TGT_VOCAB_SIZE, smoothing=0.1, ignore_index=PAD_IDX)
```

Label Smoothing 是一種 Regulation，用於改善模型的 generalization ability 和 robustness。它的核心思想是不再使用硬性的 one-hot encoding 作為目標，而是將一部分概率分配給非目標類別。

為什麼使用 Label Smoothing：

- 防止過度自信：傳統的交叉熵損失可能導致模型對預測過於自信，Label Smoothing 通過引入不確定性來緩解這個問題。
- 提高 generalization ability：通過鼓勵模型不要過分確信其預測，可以提高模型在未見數據上的表現。
- 處理標籤 noise：在某些情況下，訓練數據的標籤可能存在錯誤，Label Smoothing 可以減輕這種 noise 的影響。

LabelSmoothingLoss 的實現細節：

- confidence = 1.0 - smoothing：正確類別的置信度。
- smoothing：分配給其他類別的概率。
- cls：類別總數，這裡是目標詞彙表的大小。
- ignore_index：忽略的索引，通常用於填充標記（PAD）。

前向傳播過程：

- 將模型的預測轉換為對數概率（log_softmax）。
- 創建平滑化的目標分佈：
 - 所有類別初始化為 $\text{smoothing} / (\text{cls} - 1)$ 。
 - 正確類別的概率設置為 confidence。
- 計算平滑化目標和預測之間的 KL 散度。

● Optimizer 上使用了 Adam

```
optimizer = torch.optim.Adam(transformer.parameters(), lr=0.0001, betas=(0.9, 0.98), eps=1e-9)
```

基本概念：Adam 是一種自適應學習率優化算法，結合了 AdaGrad 和 RMSProp 的優點。它為每個參數計算自適應的學習率。

工作原理：

- 計算梯度的指數移動平均。
- 計算梯度平方的指數移動平均。
- 利用這兩個平均值來調整每個參數的學習率。

參數解釋：

- transformer.parameters(): 指定要優化的參數，這裡是整個 transformer 模型的參數。
- lr=0.0001: 初始學習率，設置為 $1e-4$ ，這是一個相對較小的學習率。
- betas=(0.9, 0.98): 用於計算梯度及其平方的運行平均值的係數。
 - 第一個值（0.9）控制動量項。
 - 第二個值（0.98）控制梯度的無中心二階矩估計。
- eps=1e-9: 添加到分母以提高數值穩定性的小常數。

策略優勢：

- 自適應學習率：為每個參數自動調整學習率，適應性強。
- 克服稀疏梯度：在處理稀疏梯度時表現良好。
- 抵消偏差：通過偏差修正來抵消初始化時的偏差。

在翻譯任務中的應用：

- 機器翻譯等 NLP 任務通常涉及大量參數和複雜的優化景觀。
- Adam 能夠有效處理這種複雜性，幫助模型更快地收斂。

● Scheduler 上使用了 ReduceLROnPlateau

```
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=10, verbose=True)
```

基本概念：

ReduceLROnPlateau 是一種自適應學習率調整策略，它會監控指定的指標（通常是驗證損失），並在該指標停止改善時降低學習率。

工作原理：

- 當被監控的指標在一定數量的 epoch 內沒有改善時，學習率會被降低。
- 這種方法有助於在訓練過程中找到更優的局部最小值。

參數解釋：

- optimizer: 要調整的優化器。
- mode='min': 指示調度器應該監控的是最小化還是最大化的指標。這裡設置為 'min'，適用於監控損失值。
- factor=0.1: 學習率降低的比例。每次觸發時，當前學習率會乘以這個因子。
- patience=10: 容忍的 epoch 數量。如果指標在這麼多 epoch 內沒有改善，則降低學習率。

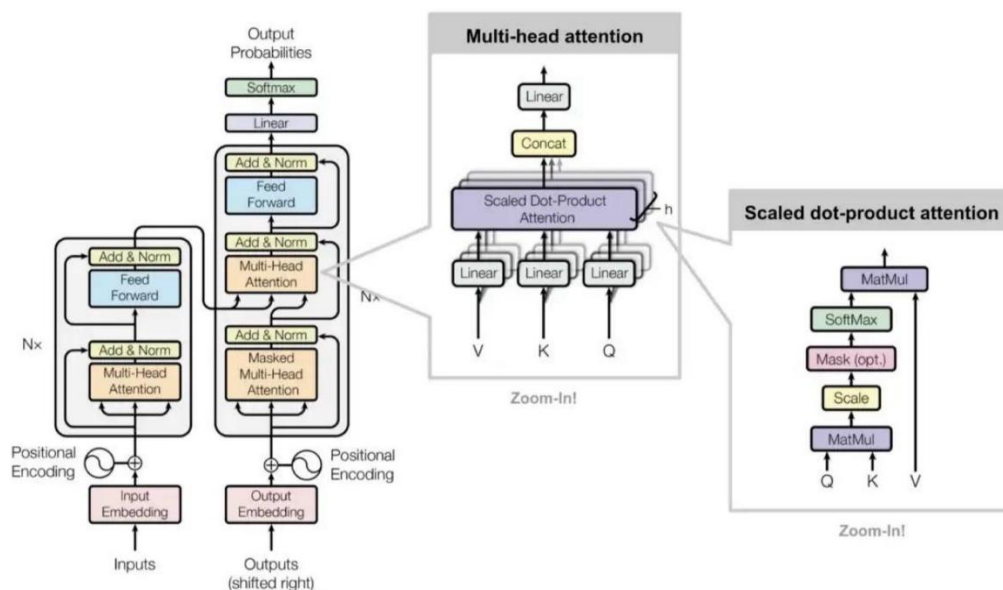
策略優勢：

- 自適應性：根據訓練進展自動調整學習率，無需手動干預。
- 優化收斂：在訓練後期幫助模型更精細地調整參數。
- 克服停滯：當模型陷入局部最小值時，降低學習率可能幫助跳出。

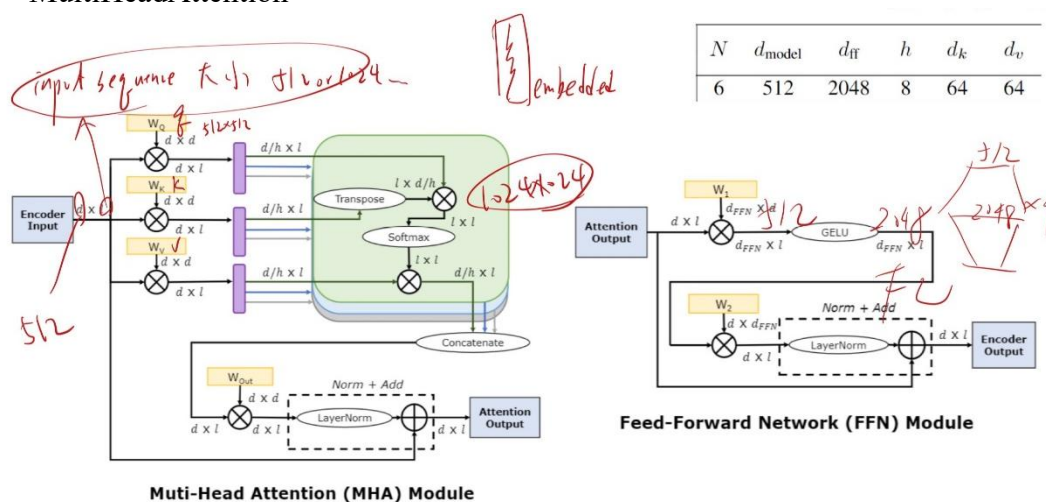
在翻譯任務中的應用：

- 對於複雜的序列到序列任務（如機器翻譯），模型的學習過程可能會經歷多個階段。
- 初期可能需要較大的學習率快速學習，後期則需要較小的學習率微調。

Model architecture:



● MultiHeadAttention



首先是劃 Multi-head attention 的部分

```

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, head_num):
        super().__init__()
        self.d_model = d_model
        self.head_num = head_num
        self.head_dim = d_model // head_num
        # 創建線性圖來轉換輸入到查詢(Q)、鍵(K)和值(V)
        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)

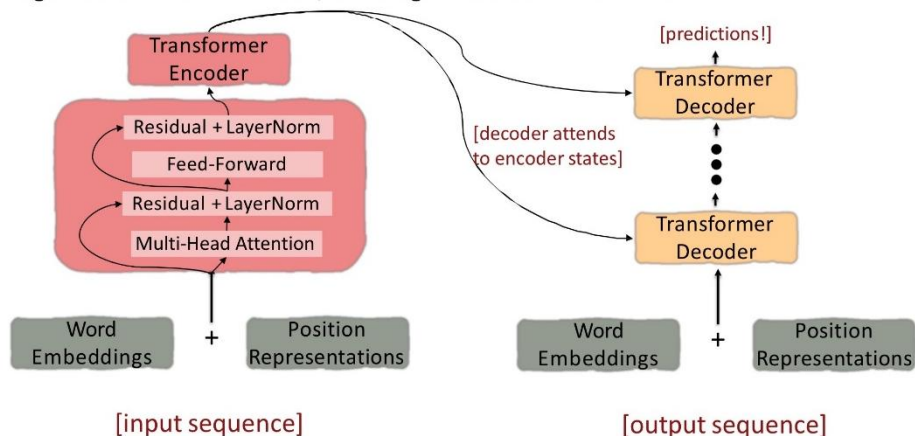
        self.out_linear = nn.Linear(d_model, d_model)
        self.layer_norm = nn.LayerNorm(d_model)

    def forward(self, Q, K, V, src_padding_mask=None, future_mask=None):
        q_seq_len, batch_size, _ = Q.shape # 獲取輸入的維度信息
        k_seq_len = K.shape[0]
        enc_inp = Q # 保存輸入用於殘差連接
        Q = self.q_linear(Q) # 通過線性圖轉換輸入
        K = self.k_linear(K)
        V = self.v_linear(V)
        # 重塑並轉置張量以準備多頭注意力計算、最終形狀: (batch_size, head_num, seq_len, head_dim)
        Q = Q.view(q_seq_len, batch_size, self.head_num, self.head_dim).transpose(0, 1).transpose(1, 2)
        K = K.view(k_seq_len, batch_size, self.head_num, self.head_dim).transpose(0, 1).transpose(1, 2)
        V = V.view(k_seq_len, batch_size, self.head_num, self.head_dim).transpose(0, 1).transpose(1, 2)
        # 計算注意力分數
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.head_dim)
        # 應用源序列的填充遮罩
        if src_padding_mask is not None:
            src_padding_mask = src_padding_mask.unsqueeze(1).unsqueeze(2).expand(-1, self.head_num, q_seq_len, -1)
            scores = scores.masked_fill(src_padding_mask, float('-inf')) # 將遮罩位置的分數設為負無窮大
        if future_mask is not None:
            future_mask = future_mask.unsqueeze(0).unsqueeze(1).expand(batch_size, self.head_num, -1, -1) # 擴展遮罩
            future_mask = future_mask.to(torch.bool) # 確保future_mask是布尔类型
            scores = scores.masked_fill(future_mask, float('-inf')) # 將遮罩位置的分數設為負無窮大
        # 應用softmax來獲得注意力權重
        attention = F.softmax(scores, dim=-1)
        output = torch.matmul(attention, V) # 將注意力權重應用到值(V)上
        output = output.transpose(1, 2).contiguous().view(batch_size, q_seq_len, self.d_model).transpose(0, 1) # 重塑
        output = self.out_linear(output) # 通過輸出線性圖
        output = self.layer_norm(output) # 應用層歸一化
        return output + enc_inp # 添加殘差連接並返回

```

● TransformerEncoder

Looking back at the whole model, zooming in on an Encoder block:




```

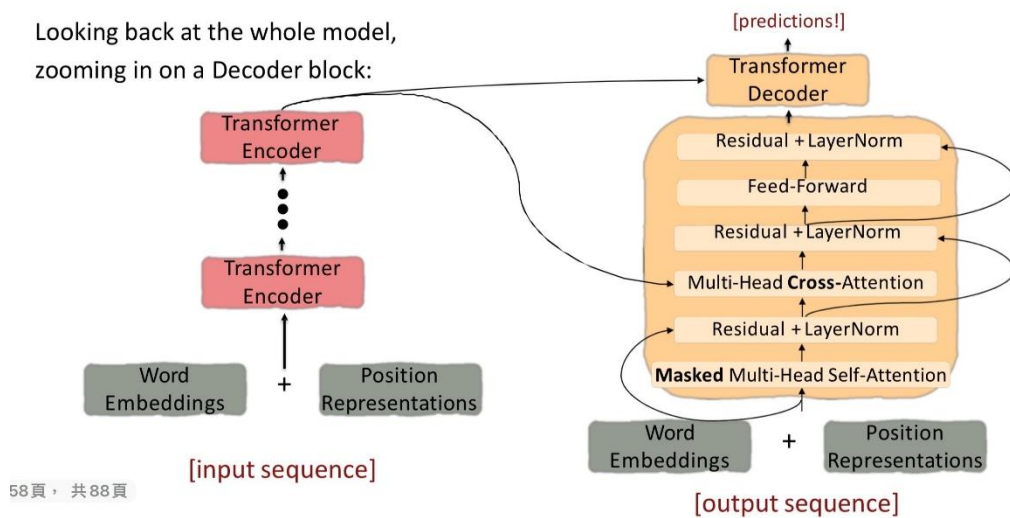
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model, dim_feedforward, nhead, dropout):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, nhead)
        self.feed_forward = nn.Sequential(
            nn.Linear(d_model, dim_feedforward), # 第一個線性層，將維度從 d_model 擴展到 dim_feedforward
            nn.GELU(), # GELU 激活函數，相比 ReLU 有更好的性能
            nn.Linear(dim_feedforward, d_model) # 第二個線性層，將維度從 dim_feedforward 縮小回 d_model
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, src_padding_mask):
        attn_output = self.self_attn(x, x, x, src_padding_mask=src_padding_mask) # 將 x 作為查詢(Q)、鍵(K)和值(V) 傳入自注意力層
        x = self.norm1(x + self.dropout(attn_output)) # 添加殘差連接 (x +)，應用 dropout，然後進行層歸一化
        ff_output = self.feed_forward(x)
        x = self.norm2(x + self.dropout(ff_output))
        return x

```

● TransformerDecoder

Looking back at the whole model,
zooming in on a Decoder block:



```

class TransformerDecoderLayer(nn.Module):
    def __init__(self, d_model, dim_feedforward, nhead, dropout):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, nhead) # 創建多頭自注意力層，用於目標序列的自注意力
        self.cross_attn = MultiHeadAttention(d_model, nhead) # 創建多頭交叉注意力層，用於關注編碼器的輸出
        self.feed_forward = nn.Sequential(
            nn.Linear(d_model, dim_feedforward),
            nn.GELU(),
            nn.Linear(dim_feedforward, d_model)
        )
        self.norm1 = nn.LayerNorm(d_model) # 用於自注意力輸出後的歸一化
        self.norm2 = nn.LayerNorm(d_model) # 用於交叉注意力輸出後的歸一化
        self.norm3 = nn.LayerNorm(d_model) # 用於前饋網絡輸出後的歸一化
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, enc_output, src_padding_mask=None, tgt_padding_mask=None, tgt_future_mask=None):
        attn_output = self.self_attn(x, x, x, src_padding_mask=tgt_padding_mask, future_mask=tgt_future_mask) # 自注意力層
        x = self.norm1(x + self.dropout(attn_output))

        attn_output = self.cross_attn(x, enc_output, enc_output, src_padding_mask=src_padding_mask) # 交叉注意力層
        x = self.norm2(x + self.dropout(attn_output))

        ff_output = self.feed_forward(x)
        x = self.norm3(x + self.dropout(ff_output))
        return x

```

● Masking

Masking the future in self-attention

- To use self-attention in **decoders**, we need to ensure we can't peek at the future.

- At every timestep, we could change the set of **keys and queries** to include only past words. (Inefficient!)

- To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$.

For encoding these words

$$e_{ij} = \begin{cases} q_i^T k_j, & j < i \\ -\infty, & j \geq i \end{cases}$$



```
def create_mask(src, tgt):
    src_seq_len, batch_size = src.shape
    tgt_seq_len = tgt.shape[0]

    src_padding_mask = (src == PAD_IDX).transpose(0, 1)
    tgt_padding_mask = (tgt == PAD_IDX).transpose(0, 1)

    tgt_future_mask = torch.triu(torch.ones((tgt_seq_len, tgt_seq_len), device=DEVICE) == 1).transpose(0, 1)
    tgt_future_mask = tgt_future_mask.float().masked_fill(tgt_future_mask == 0, float('-inf')).masked_fill(tgt_future_mask == 1, float(0.0))

    return tgt_future_mask, src_padding_mask, tgt_padding_mask
```

1.Upper Triangle (torch.triu):

- `Torch.triu(torch.ones((tgt_seq_len, tgt_seq_len), device=DEVICE) == 1)` 創建一個 True 值（指示應遮罩的位置）和 False（指示可以關注自身或先前標記的位置）的上三角矩陣。
- 此矩陣限制每個 token “看到”序列中的未來 token，強制執行 causal masking（僅關注過去和當前的 token）。

2.Masking Values:

- `masked_fill(tgt_future_mask == 0, float('-inf'))` 將下三角形和對角線設置為 0.0（以 `tgt_future_mask`），這意味著這些位置是可訪問的。
- `masked_fill(tgt_future_mask == 1, float(0.0))` 將上三角形設置為 $-\infty$ ，這意味著將來的位置無法訪問。

3.Final Values of `tgt_future_mask`:

- 下三角形和對角線：0.0（未遮罩，因此當前標記可以關注自身和以前的標記）。
- 上三角形： $-\infty$ （已遮罩，因此在解碼期間無法訪問未來的 token）。